

ETHICAL HACKING ASSIGNMENT

Reconnaissance & OWASP Top 10 Assessment

Student Name: Kamal Bahadur Basnet

Course: Cyber Security

Mentor: Sugam Dangal

Submission Date: July 13, 2026

Table of Contents

1. Executive Summary
2. Scope
3. Methodology
4. Reconnaissance
5. Tools Used
6. OWASP Top 10 Analysis
7. Findings & Recommendations
8. Conclusion
9. References

Executive Summary

This assessment was conducted as part of an educational ethical hacking assignment against an authorized target obtained through the HackerOne platform under its Responsible Disclosure Program. The main objective was to perform reconnaissance, identify technologies, discover potential attack surfaces, and map the findings to the OWASP Top 10 vulnerabilities.

Only passive and authorized reconnaissance techniques were performed. No destructive attacks or unauthorized exploitation were conducted.

Scope

Target

*.discover.com

In Scope

- Web Application
- Subdomains
- Public Assets

Out of Scope

- Physical Testing
- Social Engineering
- Denial of Services (DoS)/Distributed Denial of Services (DDoS) Attacks
- Resource Exhaustion Attacks

Methodology

The assessment followed the standard penetration testing methodology.

- Information Gathering
- Subdomain Enumeration
- Live Hosts Detection
- Port Scanning
- Technology Fingerprinting
- Historical URL Collection
- Potential Vulnerability URL Filtering
- Vulnerability Testing
- OWASP Top 10 Mapping
- Reporting

Reconnaissance

1. Web Technology Identification

The process of determining the technologies used by a website, such as the web server, programming language, content management system (CMS), JavaScript frameworks, and security mechanisms. This helps understand the target's technology stack during reconnaissance.

Wappalyzer

TECHNOLOGIES

MORE INFO

 Export

Analytics

 [Adobe Analytics](#)

Font scripts

 [Font Awesome](#)

Miscellaneous

 [ServiceNow](#)

CDN

 [Akamai](#)

Advertising

 [Twitter Ads](#)

JavaScript libraries

 [jQuery UI](#) 1.11.4

 [jQuery](#) 1.11.1

 [Boomerang](#)

UI frameworks

 [Bootstrap](#) 3.3.4

Cookie compliance

 [CrownPeak Universal Consent Platform](#)

RUM

 [Boomerang](#)

 [Akamai mPulse](#)

Whatweb

Command: whatweb https://discover.com

```
kali@kali:~/Bug_Bounty/discover$ whatweb https://discover.com
https://discover.com [301 Moved Permanently] Akamai-Global-Host, Cookies[DCID], Country[UNITED STATES][US], HTTPServer[AkamaiG
Host], HttpOnly[DCID], IP[104.69.172.81], RedirectLocation[https://www.discover.com/invalid_login/ksd/discovercardaccountcente
rlogin_discovercard.htm], UncommonHeaders[server-timing,akamai-grn]
https://www.discover.com/invalid_login/ksd/discovercardaccountcenterlogin_discovercard.htm [503 Service Unavailable] Akamai-Gl
obal-Host, Cookies[DCID], Country[UNITED STATES][US], HTTPServer[AkamaiGHost], HttpOnly[DCID], IP[104.108.247.235], Strict-Tra
nsport-Security[max-age=31536000], Title[Service Unavailable - Fail to connect], UncommonHeaders[server-timing,akamai-grn]
```

Findings

Item	Result
Web server	AkamaiGHost
Programming Language	Not Found
Framework	Bootstrap 3.3.4
CMS	ServiceNow

2. Subdomain Enumeration

Multiple subdomain enumeration tools were used to maximize discovery coverage. Individually, Subfinder identified 933 subdomains, VirusTotal identified 230, Assetfinder identified 68, and Findomain identified 666. Since many subdomains overlapped across these tools, the outputs were merged and processed using the Linux sort -u command. This removed duplicate entries and produced a final list of **1,085 unique subdomains** for further analysis.

Assetfinder

```
(kali@kali)-[~/Bug_Bounty/discover]
$ assetfinder discover.com > assetfinder.txt
```

Findomain

```
(kali@kali)-[~/Bug_Bounty/discover]
$ findomain -t discover.com -u findomain.txt
```

Target ==> discover.com

Searching in the CertSpotter API ...
Searching in the Crtsh database API ...
Searching in the Sublist3r API ...
Searching in the Threatcrowd API ...
Searching in the AnubisDB API ...
Searching in the Urlscan.io API ...
Searching in the Threatminer API ...

Subfinder

```
(kali@kali)-[~/Bug_Bounty/discover]
$ subfinder -d discover.com -o subfinder.txt
```

projectdiscovery.io

```
[INF] Current subfinder version v2.14.0 (latest)
[INF] Loading provider config from /home/kali/.config/subfinder/provider-config.yaml
[INF] Enumerating subdomains for discover.com
```

Sort -u

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ sort -u assetfinder.txt subfinder.txt virustotal.txt findomain.txt >> unique_subdomains.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l unique_subdomains.txt
1085 unique_subdomains.txt
```

3. Resolve Subdomain & IP Addresses

The process of converting discovered subdomains into their corresponding IP addresses using DNS lookups. This helps identify the servers hosting different services and reveals the target's network infrastructure.

Dnsx tool

For resolved subdomains only

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ dnsx -l unique_subdomains.txt -silent > resolved_subs.txt
```

For resolved IPs only

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ dnsx -l unique_subdomains.txt -a -resp-only > resolved_ips.txt
```

For resolved subdomains with theirs IPs

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ dnsx -l unique_subdomains.txt -a -resp -silent > resolved.txt
```

4. Live Hosts

The process of checking which discovered domains or subdomains are currently active and responding to HTTP/HTTPS requests. This helps focus security testing on accessible and operational assets.

Httpx

```
(kali㉿kali)-[~/Bug_Bounty/discover]  
$ httpx -l resolved_subs.txt -o live_subs.txt
```

5. Port Scan

Port Enumeration: Masscan was initially used to perform a high-speed scan on some most important ports and identify open ports on the target. The discovered open ports were then scanned using Nmap to verify the results and gather detailed information about the running services, service versions, and other relevant details.

Masscan

```
(kali㉿kali)-[~/Bug_Bounty/discover]  
$ sudo masscan -iL resolved_ips.txt -p 21,22,23,25,53,80,110,143,443,445,2082,2083,2086,2087,2095,2096,8000,8080,8443,8888 --rate 100 -oG masscan.txt  
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2026-07-12 16:20:56 GMT
```


Nmap

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ nmap -iL live_subs.txt -sV -O -p 21,25,53,80,110,143,443,2082,2083,2086,2095,2096 -o nmapresult.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-12 12:49 -0400
```

6. Archived URLs

The process of retrieving historical URLs for a target domain using the waybackurls tool. It gathers URLs from sources such as the Internet Archive's Wayback Machine and other public archives.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat live_subs.txt | waybackurls >> archived_urls.txt
```

7. Vulnerability Filtering

XSS

Vulnerability that allows attackers to inject malicious scripts into web pages, which execute in other users' browsers.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf xss > vuln_xss.txt
File Saved: vuln_xss.txt
(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_xss.txt
134643 vuln_xss.txt
```

SQLi

Vulnerability where malicious SQL queries are inserted into user input to access, modify, or delete database information.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf sql > vuln_sql.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_sql.txt
114972 vuln_sql.txt
```

SSRF

Vulnerability that allows an attacker to make a server send requests to internal or external resources on the attacker's behalf.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf ssrf > vuln_ssrf.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_ssrf.txt
24455 vuln_ssrf.txt
```

Open Redirect

Vulnerability that allows attackers to redirect users to malicious websites by manipulating URL parameters.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf redirect > vuln_redirect.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_redirect.txt
25540 vuln_redirect.txt
```

Path Traversal

Vulnerability that allows attackers to access files and directories outside the intended location by manipulating file paths.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf path-traversal > vuln_pathtraversal.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_pathtraversal.txt
84147 vuln_pathtraversal.txt
```

IDOR

An access control vulnerability that allows a user to access or modify another user's data by changing object identifiers, such as IDs in URLs.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf idor > vuln_idor.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_idor.txt
113953 vuln_idor.txt
```

File Upload

It occurs when an application fails to properly validate uploaded files, allowing attackers to upload malicious files that may compromise the server.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf upload > vuln_fileupload.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_fileupload.txt
70884 vuln_fileupload.txt
```

LFI

Vulnerability that allows an attacker to include or read files from the local server by manipulating file path parameters.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf lfi > vuln_lfi.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_lfi.txt
21591 vuln_lfi.txt
```

RCE

A critical vulnerability that allows attackers to execute arbitrary commands or code on a remote server.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf rce > vuln_rce.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_rce.txt
421 vuln_rce.txt
```

JWT- related endpoints

A web application endpoint that issue, validate, refresh, or revoke **JSON Web Tokens (JWTs)** for user authentication and authorization.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ cat archived_urls.txt | gf jwt > vuln_jwt.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l vuln_jwt.txt
98567 vuln_jwt.txt
```

8. Vulnerability Testing

SQLMap

An open-source penetration testing tool that automates the detection & exploitation of SQL injection(SQLi) vulnerabilities in web applications.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ sqlmap -m vuln_sql_i.txt --batch --random-agent --level=5
```

No SQL Injection Found.

Dalfox

An open-source tool used to detect and verify Cross-Site Scripting (XSS) vulnerabilities in web applications.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ dalfox file vuln_xss.txt -o dalfox_results.txt
```

No XSS vulnerability detected.

Nuclei

A fast, template-based vulnerability scanner widely used during authorized penetration tests and bug bounty programs.

```
(kali㉿kali)-[~/Bug_Bounty/discover]
nuclei -l live_subs.txt -sa -o nucleireresults.txt -silent -timeout 4
```

No Vulnerability detected.

9. Searching for Interesting Endpoints

Login Pages

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ grep -iE "login|signin|auth|account/login|user/login" archived_urls.txt > login_pages.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l login_pages.txt
10975 login_pages.txt
```

Admin Panels

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ grep -iE "admin|administrator|adminpanel|cpanel|dashboard|manage" archived_urls.txt > admin_panels.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l admin_panels.txt
2351 admin_panels.txt
```

API Endpoints

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ grep -iE "/api/|api\.|graphql|rest|v1|v2|swagger|openapi" archived_urls.txt > api_endpoints.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l api_endpoints.txt
67544 api_endpoints.txt
```

JavaScript files

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ grep "\.js" archived_urls.txt > js_files.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l js_files.txt
105015 js_files.txt
```

Backup Files

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ grep -iE "\.(bak|backup|old|zip|tar|gz|rar|7z|sql|db|tgz)$" archived_urls.txt > backup_files.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l backup_files.txt
0 backup_files.txt
```

Configuration Files

```
(kali㉿kali)-[~/Bug_Bounty/discover]
$ grep -iE "\.env|config|configuration|settings|web\.config|config\.php|config\.json|config\.yaml|config\.yml|config\.xml" archived_urls.txt > config_files.txt

(kali㉿kali)-[~/Bug_Bounty/discover]
$ wc -l config_files.txt
17679 config_files.txt
```

Tools Used

Tool name	Purpose
Subfinder, Virustotal, Assetfinder, Findomain	Discover Subdomains
Dnsx	Resolved subdomains & IPs
Httpx	Check live hosts
Masscan/ Nmap	Port scanning
Whatweb	Technology detection
Wappalyzer	Web technology identification
Gf	Filter vulnerable parameters
SQLMap	SQL injection testing
Dalfox	XSS testing
Nuclei	Vulnerability scanning

OWASP Top 10 Analysis

A01 Broken Access Control

Description

Users can access data or perform actions they are not authorized to access.

Causes

- Missing or weak authorization checks
- Improper permission validation
- Insecure access control logic

Common vulnerabilities

- IDOR
- Privilege Escalation
- BOLA
- Forced Browsing
- SSRF

How to remediate

- Enforce server-side authorization
- Follow the least privilege principle
- Deny access by default

A02 Security Misconfiguration

Description

The application or server is configured insecurely, making it easier for attackers to exploit.

Causes

- Default settings
- Exposed admin pages
- Unnecessary services
- Missing security headers

Common vulnerabilities

- Defaults Credentials
- Directory Listing
- Debug Mode Enabled
- Sensitive Cookie Without Secure Attribute
- Password in Configuration File

How to remediate

- Remove default configurations
- Disable unused services
- Apply security headers
- Perform regular configurations reviews

A03 Software Supply Chain Failures

Description

Security risks are introduced through vulnerable or compromised third-party software and dependencies.

Causes

- Outdated libraries
- Vulnerable dependencies
- Insecure CI/CD pipelines
- Compromised third-party packages

Common Vulnerabilities

- Vulnerable dependencies
- Dependency Confusion
- Malicious Packages
- Typosquatting
- CI/CD Pipeline Compromise

How to remediate

- Keep dependencies updated
- Verify package integrity
- Use trusted repositories
- Secure CI/CD pipelines

A04 Cryptographic Failures

Description

Sensitive information is not properly protected because encryption is weak, missing, or misused.

Causes

- Weak encryption
- Poor key management
- Storing sensitive data in plain text
- Insecure TLS configuration

Common Vulnerabilities

- Weak Encryption
- Hard-coded keys
- Weak Password Hashing
- Sensitive Data Exposure
- Insecure TLS

How to remediate

- Use strong encryption
- Secure key management
- Encrypt sensitive data
- Hash Passwords with Argon2 or bcrypt

A05 Injection

Description

The application treats untrusted user input as commands or code, allowing attackers to manipulate its behavior.

Causes

- Unsanitized user input
- Improper input validation
- Unsafe query construction

Common Vulnerabilities

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- NoSQL Injection
- LDAP Injection

How to remediate

- Use parameterized queries
- Validate inputs
- Encode outputs
- Avoid dynamic queries

A06 Insecure Design

Description

Security was not considered during the design phase, leaving the application with built-in weaknesses.

Causes

- Lack of security planning
- Missing threat modeling
- Insecure business logic
- Poor architecture

Common vulnerabilities

- Business Logic Flaws
- Missing Rate Limiting
- Weak Password Policies
- Insecure Workflows
- Design Flaws

How to remediate

- Perform threat modeling
- Define security requirements early
- Follow secure design principles
- Conduct architecture reviews

A07 Authentication Failures

Description

Weak authentication allows attackers to log in as legitimate users.

Causes

- Weak passwords
- Poor session management
- Missing MFA
- Insecure authentication mechanisms

Common vulnerabilities

- Brute Force
- Credential Stuffing
- Session Hijacking
- Session Fixation
- Weak Password Policy

How to remediate

- Enable MFA
- Enforce strong passwords
- Secure sessions
- Monitor login attempts

A08 Software or Data Integrity Failures

Description

The application cannot verify that its software or data has remained unchanged and trustworthy.

Causes

- Missing integrity checks
- Insecure software updates
- Unsigned code
- Insecure deserialization

Common Vulnerabilities

- Insecure Deserialization
- Unsigned Updates
- Package Tampering
- Code Injection
- CI/CD Integrity Issues

How to remediate

- Verify digital signatures
- Validate software updates
- Secure build pipelines
- Only use trusted code sources

A09 Security Logging and Alerting Failures

Description

Security incidents go unnoticed because important events are not properly logged or monitored.

Causes

- Missing logs
- Poor monitoring
- Inadequate alerting
- Weak incident detection

Common vulnerabilities

- Missing Auditing Logs
- Insufficient Monitoring
- Missing Alerts
- Log Tampering
- Poor Incident Response

How to remediate

- Log important security events
- Monitor logs continuously
- Configure alerts
- Establish an incident response plan

A10 Mishandling of Exceptional Conditions

Description

The application handles unexpected errors insecurely, creating opportunities for attackers to exploit.

Causes

- Poor exception handling
- Fail-open behavior
- Missing input validation
- Improper error handling

Common vulnerabilities

- Information Disclosure
- Fail-Open Authentication
- Null Pointer Errors
- Unhandled Exceptions
- Denial of Service (DOS)

How to remediate

- Handle exceptions securely
- Fail safely
- Sanitize error messages
- Validate Inputs

Findings & Recommendations

Executive Findings

During the reconnaissance phase, multiple publicly accessible assets associated with the target were identified. A combination of passive and active reconnaissance techniques was used to enumerate subdomains, identify live hosts, discover open ports, fingerprint technologies, and collect historical URLs. The collected data helped identify the target's public attack surface for further security analysis.

Summary of Findings

Findings	Result
Technologies Identified	AkamaiGhost, Bootstrap 3.3.4, ServiceNow
Total Unique Subdomains	1085
Open Ports	21, 25, 53, 110, 143, 443, 2082, 2083, 2086, 2087, 2095, 2096
Archived URLs	Successfully collected.
Login Pages	Identified
API Endpoints	Identified
JAVAScript Files	Identified
SQL Injection	Not Detected.
Cross-Site Scripting (XSS)	Not Detected.
Nuclei Scan	Not confirmed vulnerabilities detected.

Recommendations

- Regularly review & remove unused subdomains.
- Protect administrative interfaces using Multi-Factor Authentication (MFA) and strong passwords
- Secure API endpoints with proper authentication and authorization.
- Keep web technologies and third-party components up to date.
- Review publicly accessible historical URLs for sensitive information.

Overall Risk Rating

Severity	Findings
Critical	None identified.
High	None identified.
Medium	Administrative interface URLs were identified and should be protected with strong authentication and access controls.
Low	Public subdomains, login pages, JavaScript files, and archived URLs were identified.
Informational	Technologies, open ports, live hosts, were successfully identified.

Conclusion

The assessment successfully identified the target's public attack surface through reconnaissance techniques, including subdomain enumeration, live host detection, port scanning, technology fingerprinting, historical URL collection, and endpoint identification. A total of 1,085 unique subdomains were discovered after combining results from multiple enumeration tools and removing duplicate entries. No confirmed SQL Injection, Cross-Site Scripting (XSS), or other exploitable vulnerabilities were identified during the authorized testing. The recommendations provided in this report aim to improve the overall security posture of the target by reducing unnecessary exposure and strengthening defensive controls.

References

- OWASP Top 10 (2025)
- Project Discovery Documentation
- VirusTotal
- Wappalyzer
- HackerOne